

DATA 266: Generative AI, Fall 2026

Lab 1 Report

Character-Level Language Model Pretraining, Sentiment Classification,
and CycleGAN Image Style Transfer

Team 09 (PairProgramming_Team_09)

Shibin Thomas
Denisha Ketan Tank

Repository: <https://github.com/Shibin-1020/Gen-AI---LAB-1>

Branch: shibin-lab1

October 2026

1 Team Ownership Statement

Shibin Thomas built the character-level GPT for Task 1, the mean-pooling baseline, TextCNN and BiGRU-with-attention classifiers for Task 2, and the CycleGAN for Task 3, including the Kaggle submission. Shibin also set up the shared TinyStories download, the shared metric modules for Tasks 1 and 2, the repository layout, the run manifests and the smoke tests. Denisha Ketan Tank built an independent character-level GPT for Task 1, a mean-pooling baseline, TextCNN and bidirectional GRU classifiers for Task 2, and a CycleGAN for Task 3, together with the team's best Kaggle submission and its official evaluation. Each member trained their own models from scratch in a personal folder for every task (task1_llm/<member>, task2_sentiment/<member>, task3_gan/<member>). To keep the results comparable we shared the raw TinyStories file and the generation prompts for Task 1, the official Yelp Polarity test split and a common metric module for Task 2, and the instructor's evaluation script for Task 3. The comparison tables and the analysis sections of this report were written jointly.

1.1 Repository Organisation and Reproducibility

Every member folder follows the same layout: source code and the task notebook in `src/`, configuration files in `configs/`, model weights in `checkpoints/<run_id>/`, plots and samples in `outputs/<run_id>/`, and the files `metrics_report.csv`, `results.md` and `failure_analysis.md`. Unedited training logs are kept in `reproducibility/raw_logs/` and a manifest for each run (configuration, git commit, library versions, hardware and checkpoint hashes) in `reproducibility/manifests/`. Each figure and table below names the file it was taken from, so every number can be traced back to its log or checkpoint.

2 Task 1: GPT-Style Character-Level Language Model

The model is a decoder-only Transformer trained on characters from TinyStories [2]. All components (token and positional embeddings, layer normalisation, multi-head causal self-attention, the feed-forward blocks and the residual connections) were written by hand following Vaswani et al. [1], with the pre-LayerNorm arrangement of Xiong et al. [5] and the GPT-2 initialisation scheme [4]; no built-in Transformer or attention module of PyTorch is used, and a unit test (`tests/test_model.py`) checks this. Each member drew their own split from the cleaned story pool. Shibin's split contains 100,000 training and 10,000 validation stories (seed 266), and the character vocabulary of 90 symbols was built from the training split only. The network has 6 layers, a model width of 320, 8 attention heads and a context of 256 characters, about 7.5 million parameters in total. Denisha built a smaller and shallower model (4 layers, width 256, 4 heads, context 128, 3.25 million parameters) on her own 100,000/10,000 split (seed 20260929) and trained it on a Tesla T4 with fp16 mixed precision, so the two models differ in depth, width, context length and optimiser settings.

2.1 Model Comparison

Table 1. Task 1 comparison of architecture, hyperparameters and metrics. Losses, perplexity, BPC and accuracy are computed on the best checkpoint; metric definitions follow the shared module `task1_llm/shared_eval/metrics.py`.

	Shibin Thomas (gpt_char_v1)	Denisha Ketan Tank (char GPT)
Architecture	6 layers, d_model 320, 8 heads (head dim 40), FFN 4x with GELU, pre-LayerNorm, learned positions, tied input/output embeddings, context 256	4 layers, d_model 256, 4 heads (head dim 64), FFN 1024 with GELU, pre-LayerNorm, learned positions, context 128, vocabulary 114
Hyperparameters	AdamW (0.9, 0.95), weight decay 0.1, peak LR 0.0006 with 2% warm-up and cosine decay to 6e-05, batch 64 x 256, gradient clip 1.0, dropout 0.1, 10 epochs, bf16	AdamW (0.9, 0.999), weight decay 0.01, peak LR 5e-4 with 500 warm-up steps and cosine decay to 10%, batch 48 x 128, gradient clip 1.0, dropout 0.1, 10 epochs, fp16 AMP
Training cross-entropy (nats/char)	0.5594	0.7128
Validation cross-entropy (nats/char)	0.5711	0.6822
Validation perplexity	1.7701	1.9783
Validation bits per character	0.8239	0.9842
Training perplexity / BPC	1.7497 / 0.8071	2.0397 / 1.0283
Generalisation gap (val - train CE)	0.0116	-0.0306
Next-character accuracy (validation)	0.8173	0.7822
Distinct-1 / 2 / 3 (sampled, T = 0.8)	0.2277 / 0.7048 / 0.9264	0.1080 / 0.4582 / 0.7360 (T = 0.7-1.1)
Repeated 4-gram rate (sampled / greedy)	0.0066 / 0.1373	0.1526 / not run
Gradient norm mean / p99 / max	0.304 / 1.145 / 14.71	final 0.232 / max 0.287 (per epoch)
Clipped steps / loss spikes / NaN steps	1.34% / 0 / 0	not logged / not logged / 0
Parameters (total / non-embedding)	7,509,120 / 7,398,400	3,250,688 / not reported
Training throughput (characters/s)	569,917	224,658
Generation speed (characters/s)	255.0	268.0000
Peak GPU memory (MB)	3,347	408
Training time (min) / epochs / best epoch	28.9 / 10 / 10	66.6 / 10 / 10
Hardware	NVIDIA GeForce RTX 5090	NVIDIA Tesla T4 (Colab)
Run and checkpoint	gpt_char_v1_20261001-153007, best_model.pt (SHA-256 97647773d06f)	checkpoints/gpt_from_scratch.pt (SHA-256 3858840ebe56)

Source: `task1_llm/shibin_thomas/metrics_report.csv`, `task1_llm/member_denisha/outputs/metrics.csv` (branch `denisha-lab1`), `task1_llm/member_denisha/config.yaml` (branch `denisha-lab1`)

2.2 Training Behaviour

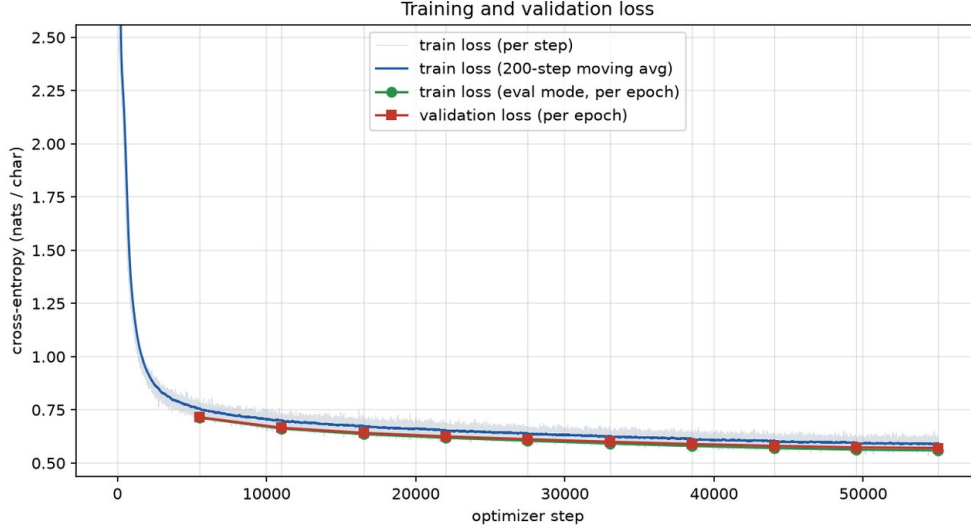


Figure 1. Task 1 training and validation loss (Shibin Thomas)
(task1_llm/shibin_thomas/outputs/gpt_char_v1_20261001-153007/loss_curves.png)

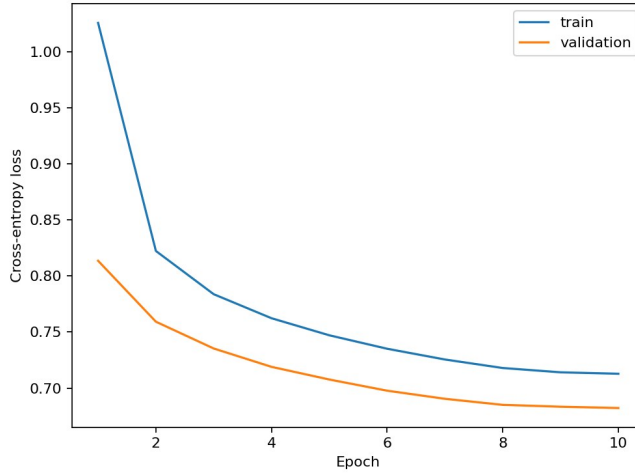


Figure 2. Task 1 training and validation loss per epoch (Denisha Ketan Tank)
(task1_llm/member_denisha/outputs/loss_curves.png)

Table 2. Validation cross-entropy (nats/char) and next-character accuracy (%) after each epoch.

Epoch	1	2	3	4	5	6	7	8	9	10
Val. CE	0.715	0.666	0.642	0.625	0.612	0.600	0.590	0.581	0.574	0.571
Val. acc.	77.4	78.8	79.6	80.1	80.5	80.8	81.2	81.4	81.6	81.7

Source: reproducibility/raw_logs/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007/epochs.csv

The training loss starts at 4.52, which is the loss of a uniform guess over 90 characters, and drops below 1.0 within the first 1,400 steps. After that the validation loss keeps falling every epoch, from 0.715 to 0.571, so the last epoch is also the best one. The gap between validation and training loss stays very small (0.012 nats at the end), which is expected with roughly 120 training characters per parameter. Training was stable: there were no NaN steps or loss spikes, and all of the 738 clipped steps happened during the learning-rate warm-up. Denisha's model followed the same pattern with a slightly negative generalisation gap (validation loss 0.682 against a training loss of 0.713 measured with dropout active), no NaN gradients and a final

gradient norm of 0.23.

2.3 Discussion

Strengths. Both small models, trained from scratch on a single GPU, write fluent TinyStories-style text with named characters, dialogue and simple cause and effect. Shibin's 7.5M-parameter model reaches a validation BPC of 0.824 and 81.7% next-character accuracy; Denisha's 3.25M-parameter model reaches 0.984 and 78.2%. The comparison shows the expected effect of capacity and context: the deeper, wider model with twice the context predicts better and also produces more varied text (Distinct-3 of 0.93 against 0.74 and a repeated 4-gram rate of 0.007 against 0.153), while the smaller model needs less than a tenth of the GPU memory (408 MB against 3,347 MB).

Weaknesses. Both models show the same three failure types: repetition, broken or invented words, and loss of coherence. Greedy decoding falls into loops (repeated 4-gram rate 0.137 for Shibin's model). Because words are produced one character at a time, rare words are misspelled and sometimes spelled differently each time they appear. Stories lose their thread once they grow beyond the context window, which happens sooner for Denisha's 128-character context, where a new story often starts in the middle of a continuation.

Limitations. Each model was trained once with a single seed, on different hardware (RTX 5090 and Tesla T4) and on different random splits, so small differences should not be over-interpreted. The diversity metrics were computed on different prompts and temperatures (10 shared prompts at $T = 0.8$ for Shibin, one prompt at $T = 0.7$ to 1.1 for Denisha). Neither validation loss had fully flattened by epoch 10, Distinct-n measures diversity rather than story quality, and generation speed was measured without a key-value cache.

Next steps. We would train for more epochs or use a slightly larger model, extend the context to 512 characters, and compare top-k or nucleus sampling with a repetition penalty on the shared prompts. A key-value cache would speed up generation, and a sub-word tokenizer trained from scratch would remove most of the spelling errors.

2.4 Failure Analysis (Shibin Thomas)

We generated text for the 10 shared prompts, with one greedy and three sampled continuations ($T = 0.8$) of 500 characters each. The three clearest failures are shown below; the full set of 40 samples is in `outputs/<run_id>/samples.md`.

Case 1: Repetition (greedy decoding)

In a small house near the forest, there was a big tree. The tree was very high and the tree was very high. The tree was very high up in the tree. The tree was very high and the tree was very high.

One day, the tree saw a big tree. The tree was very high and the tree was very high. The tree was very high and the tree was very high. ...

The phrase “the tree was very” appears 16 times and the repeated 4-gram rate of this sample is 0.61, against 0.04 on average. Greedy decoding always takes the most likely character, so once a high-probability phrase has been written, starting it again is again the most likely continuation. Sampling, or a repetition penalty, would break the loop; the repeated 4-gram rate on the same prompts is the measure to check.

Case 2: Broken and inconsistent spelling ($T = 0.8$)

Tim was scared. He ran and ran until he came across a big lute. He saw a small tree which was very pretty. Tim wanted to take the luter from him. He said yes, but he had to be careful.

The luter was big and boring. Tim saw that the lutte was scary and hurt. ...

The rare word “lute” is spelled four different ways (lute, litter, luter, lutte). The model has no word list and rebuilds a rare word from frequent letter patterns each time it comes back. A lower temperature, longer training or sub-word tokens should reduce this.

Case 3: Loss of coherence (T = 0.8)

After school, Sam wanted to go to the zoo and see all the animals.

They all walked closer and held on tight. Sam thought it was tremometer than the diamond. ... Together, Jack and Sam started to fill their bucket with food. He was so proud of their work! ... Tom and Sam continued to throw their bucket until it was time to go hom

Each sentence is fluent, but the setting jumps from the zoo to a diamond and a bucket, two new friends (Jack and Tom) appear without introduction, and the pronouns stop matching. The continuation is 500 characters long while the model only sees the last 256, so the beginning of the story has literally dropped out of its context. A longer context or a deeper model would be the fix to test.

2.5 Failure Analysis (Denisha Ketan Tank)

Denisha generated continuations of the prompt “Henry the mule was so pleased” at temperatures 0.7, 0.9 and 1.1 and analysed one failure in each.

Case 1: Repetition (T = 0.7)

Henry the mule was so pleased with his paint and he said, "You are very sweet, Lily. You are very brave and smart. You are very smart and kind."

Lily and Tom nodded. They got in the car and drove to their car. ...

The phrase “You are very” repeats three times in two sentences, and the repetition rate Denisha measured for this continuation is 0.237. “drove to their car” after “got in the car” is a local loop. Denisha's proposed test is to compare temperatures 0.7, 0.9 and 1.1 and add a repetition penalty, measuring the repeated 4-gram rate.

Case 2: Abrupt topic change (T = 0.9)

Henry the mule was so pleased that he spilled his hand.

The End.

Once there was a girl. She always wanted to explore. One day she was walking in the park and she saw the sand spraying snowball covered popcorn. ...

After one nonsensical sentence the model writes “The End.” and starts an unrelated story. With a 128-character context the original character is forgotten almost immediately, and the noun phrase “sand spraying snowball covered popcorn” is grammatical but meaningless.

Case 3: Broken grammar and coherence (T = 1.1)

Henry the mule was so pleased that he and helped her mommy cook a nice magic fort!
After that, his mummy reminded him to listen carefully and jog with his owner.
... She loved good dream about the airport. One day, she decided to use a whip to belong

At high temperature the grammar breaks (“he and helped”, “loved good dream”, “to use a whip to belong”) and the pronouns switch between him and her. Lower temperatures and longer training are the proposed fixes.

Source: task1_llm/member_denisha/failure_analysis.md (branch denisha-lab1),
task1_llm/member_denisha/outputs/samples.json (branch denisha-lab1)

2.6 Evidence

Evidence	Location in the repository
Shibin: Configuration	task1_llm/shibin_thomas/configs/gpt_char_v1.yaml
Shibin: Training log	reproducibility/raw_logs/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007/train.log
Shibin: Per-step and per-epoch logs	reproducibility/raw_logs/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007
Shibin: Run manifest	reproducibility/manifests/task1_llm/shibin_thomas/gpt_char_v1_20261001-153007.json
Shibin: Plots (loss, gradient norm, LR)	task1_llm/shibin_thomas/outputs/gpt_char_v1_20261001-153007
Shibin: Generated samples	task1_llm/shibin_thomas/outputs/gpt_char_v1_20261001-153007/samples.md
Shibin: Best checkpoint	task1_llm/shibin_thomas/checkpoints/gpt_char_v1_20261001-153007/best_model.pt
Shibin: Results and design choices	task1_llm/shibin_thomas/results.md
Shibin: Executed notebook	task1_llm/shibin_thomas/src/task1_gpt_tinystories.ipynb
Denisha: Configuration	task1_llm/member_denisha/config.yaml (branch denisha-lab1)
Denisha: Training log	reproducibility/raw_logs/task1_denisha.log (branch denisha-lab1)
Denisha: Run manifest	reproducibility/manifests/task1_denisha.json (branch denisha-lab1)
Denisha: Metrics and history	task1_llm/member_denisha/outputs/metrics.csv (branch denisha-lab1)
Denisha: Loss curves	task1_llm/member_denisha/outputs/loss_curves.png (branch denisha-lab1)
Denisha: Generated samples	task1_llm/member_denisha/outputs/samples.json (branch denisha-lab1)
Denisha: Checkpoint	task1_llm/member_denisha/checkpoints/gpt_from_scratch.pt (branch denisha-lab1)
Denisha: Results	task1_llm/member_denisha/results.md (branch denisha-lab1)
Denisha: Notebook	task1_llm/member_denisha/Task1_GPT_Colab.ipynb (branch denisha-lab1)

3 Task 2: Sentiment Classification on Yelp Polarity

All Task 2 models classify a review from the Yelp Review Polarity dataset [6] as positive or negative and learn their word embeddings from random initialisation; no pretrained embeddings or language models are used. The official test split (38,000 reviews) is shared by the team. Each member carved a validation set out of the training split; Shibin used 50,000 reviews (seed 266) for early stopping and model selection. Preprocessing removes duplicates and reviews that also appear in the test set, lowercases the text, expands contractions, removes punctuation and stopwords while keeping negations and contrast words, and applies Snowball stemming. The vocabulary of 30,000 tokens is built from the training split only.

Shibin trained three models that share the same 128-dimensional embedding, so that differences in the results come from the architecture. The baseline averages the word embeddings of a review and feeds them to a small MLP, in the spirit of fastText [8]. The first experimental model is a TextCNN [7] with filter widths 3, 4 and 5 and max pooling over time. The second is a two-layer bidirectional GRU with additive attention pooling [9].

Denisha also trained a mean-pooling baseline, a TextCNN and a two-layer bidirectional GRU with learned 128-dimensional embeddings, but with different hyperparameters: batch size 512, a reduce-on-plateau learning-rate schedule, up to 12 epochs, bfloat16 mixed precision, an MLP hidden size of 128 for the baseline, dropout 0.3 for the CNN, and a GRU classifier that reads the output at the last token instead of using attention. Her TextCNN has the same layer sizes as Shibin's, so the two differ only in their training hyperparameters. One important difference concerns evaluation: Denisha split the 560,000 official training reviews into her own training, validation and test sets and evaluated on her held-out test set of 112,000 reviews, whereas Shibin evaluated on the official test split of 38,000 reviews. Both test sets come from the same distribution and are large, but the numbers are not computed on identical reviews.

3.1 Model Comparison

Table 3. Task 2 comparison. Shibin's models are evaluated on the official test split (n = 38,000) and share: vocabulary 30K, maximum length 256 tokens (head and tail), 3% warm-up with cosine decay, weight decay 1e-4, gradient clipping 1.0, early stopping on validation macro-F1 (run yelp3_20261001-182849). Denisha's models are evaluated on her own held-out test split (n = 112,000) and share: vocabulary 30K, maximum length 256, weight decay 1e-4, early stopping on validation loss. Threshold 0.5 for all models.

	Shibin: Baseline (mean pool)	Shibin: Exp. 1 (TextCNN)	Shibin: Exp. 2 (BiGRU + attn.)	Denisha: Baseline (BoW MLP)	Denisha: Exp. 1 (TextCNN)	Denisha: Exp. 2 (BiGRU)
Architecture	Embedding 128, masked mean pooling, MLP (64), sigmoid output	Embedding 128, Conv1d widths 3/4/5 with 128 filters each, max-over-time pooling	Embedding 128, 2-layer BiGRU (128 per direction), additive attention pooling	Embedding 128, mean pooling, MLP (128), dropout 0.2	Embedding 128, Conv1d widths 3/4/5 with 128 channels, max pooling	Embedding 128, 2-layer bidirectional GRU (128 per direction), output at the last valid token
Hyperparameters	AdamW lr 2e-3, batch 512, dropout 0.3, up to 8 epochs	AdamW lr 1e-3, batch 256, dropout 0.5, up to 5 epochs	AdamW lr 1e-3, batch 256, dropout 0.3, up to 5 epochs	AdamW lr 1e-3, batch 512, dropout 0.2, up to 12 epochs, LR halved on plateau	AdamW lr 1e-3, batch 512, dropout 0.3, up to 12 epochs, LR halved on plateau	AdamW lr 1e-3, batch 512, dropout 0.3, up to 12 epochs, LR halved on plateau
Accuracy	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
Accuracy 95% CI	[0.9276, 0.9330]	[0.9446, 0.9488]	[0.9537, 0.9577]	[0.9307, 0.9337]	[0.9456, 0.9482]	[0.9507, 0.9531]
Precision (macro)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
Recall (macro)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
F1 (macro)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
Precision (micro)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
Recall (micro)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
F1 (micro)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
Precision (weighted)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
Recall (weighted)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
F1 (weighted)	0.9304	0.9467	0.9558	0.9321	0.9468	0.9519
Confusion matrix [TN FP; FN TP]	[17,718 1,282; 1,363 17,637]	[17,951 1,049; 976 18,024]	[18,029 971; 710 18,290]	[52,150 3,850; 3,755 52,245]	[53,049 2,951; 3,004 52,996]	[53,209 2,791; 2,601 53,399]
ROC-AUC	0.9800	0.9883	0.9917	0.9804	0.9883	0.9901
PR-AUC	0.9802	0.9885	0.9920	0.9797	0.9877	0.9898
MCC	0.8608	0.8934	0.9116	0.8642	0.8937	0.9037
MCC 95% CI	[0.8553, 0.8659]	[0.8891, 0.8975]	[0.9076, 0.9155]	[0.8613, 0.8673]	[0.8912, 0.8964]	[0.9015, 0.9062]
Brier score	0.0520	0.0399	0.0337	0.0510	0.0396	0.0365
ECE (15 bins)	0.0045	0.0049	0.0110	0.0064	0.0080	0.0081
McNemar vs baseline (b / c, p)	-(baseline)	613 / 1,233, p=4.67e-47	557 / 1,521, p=4.65e-99	-(baseline)	1,849 / 3,499, p=1.38e-112	1,856 / 4,069, p=1.32e-181
Parameters	3,848,321	4,037,377	4,367,873	3,856,641	4,037,377	4,334,849
Training time (min)	0.3545	0.9865	19.1767	0.4411	3.0057	2.6504
Epochs run (best)	8 (8)	5 (5)	5 (4)	12 (4)	12 (5)	12 (3)
Train examples/sec	279,909	54,074	2,271	182,819	26,829	30,426
Inference examples/sec	2,126,409	193,858	29,489	not reported	not reported	not reported
Peak GPU memory (MB)	1,149	1,277	1,392	235	431	1,717
Hardware	NVIDIA GeForce RTX 5090	NVIDIA GeForce RTX 5090	NVIDIA GeForce RTX 5090	Tesla T4	Tesla T4	Tesla T4
Checkpoint (SHA-256)	725dff759e4	b4226830a1dc	2ecf44d666cf	65cddf7326f7	1e01df581320	cbcaf6a5f0f2

Source: task2_sentiment/shibin_thomas/outputs/yelp3_20261001-182849/comparison.csv, task2_sentiment/shared_eval/metrics.py, task2_sentiment/member_denisha/metrics_report.csv (branch denisha-lab1), task2_sentiment/member_denisha/config.yaml (branch denisha-lab1)

Macro, micro and weighted scores are identical, or nearly so, because both test sets are balanced (19,000 reviews per class in the official split and 56,000 per class in Denisha's split): weighted averaging then uses

equal weights, and micro averaging always equals accuracy for a single-label binary task.

Table 4. Error rate (%) on subsets of each member's test set. Shibin's length slices are at most 50, 51 to 150 and more than 150 words ($n = 9,287 / 16,910 / 11,803$); Denisha's use her own length bins ($n = 26,869 / 50,261 / 34,870$). Denisha did not compute the negation and contrast slices.

Slice	Shibin: Baseline	Shibin: TextCNN	Shibin: BiGRU + attn.	Denisha: Baseline	Denisha: TextCNN	Denisha: BiGRU
Contains a negation	7.47	5.50	4.48	-	-	-
Contains a contrast word	7.89	5.94	4.87	-	-	-
Neither negation nor contrast	4.93	4.53	4.16	-	-	-
Short reviews	6.96	5.29	5.01	6.96	5.94	5.32
Medium-length reviews	6.94	5.22	4.09	6.69	5.11	4.67
Long reviews	7.00	5.52	4.45	6.81	5.14	4.64

Source: task2_sentiment/shibin_thomas/metrics_report.csv,
task2_sentiment/member_denisha/metrics_report.csv (branch denisha-lab1)

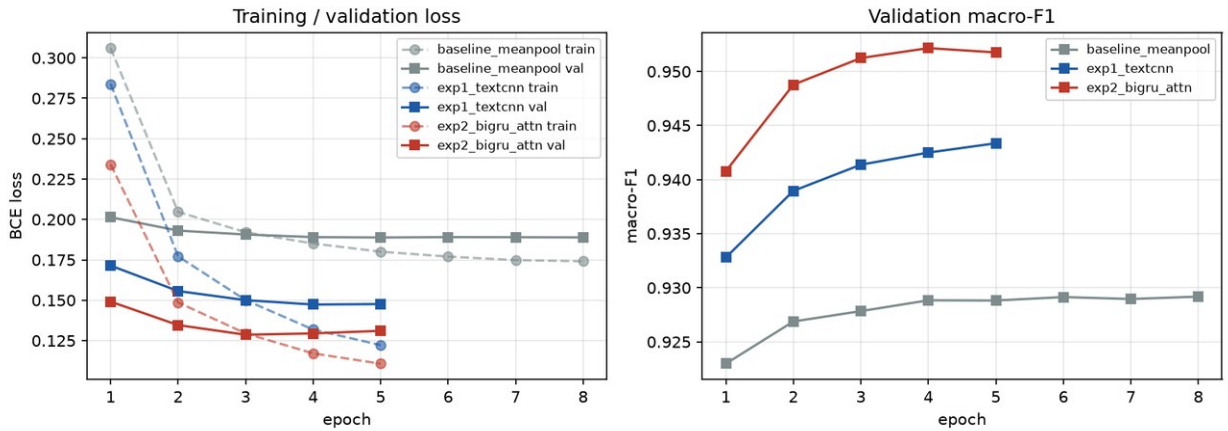


Figure 3. Training and validation loss and validation macro-F1 per epoch for the three Task 2 models (task2_sentiment/shibin_thomas/outputs/yelp3_20261001-182849/training_curves.png)

3.2 Discussion

Strengths. Both members found the same ranking, recurrent model > TextCNN > baseline, on every discrimination metric, which makes the result robust to the different splits, hyperparameters and hardware. The best models reach 95.58% (Shibin, BiGRU with attention, official test set) and 95.19% (Denisha, BiGRU, her own test set) accuracy, and the baselines agree closely (93.04% and 93.21%). For Shibin's models the 95% bootstrap intervals do not overlap and paired McNemar tests on the same 38,000 reviews give p-values below $1e-20$ for each pair; Denisha's McNemar tests against her baseline give p-values below $1e-110$. Compared with the baseline, the BiGRU removes 36% of all errors, and 40% and 38% of the errors on reviews containing a negation or a contrast word, which is exactly where a bag of words cannot work. All three models are well calibrated (expected calibration error [10] at most 1.1%). Since the shared embedding table holds 3.84 of the roughly 4 million parameters in every model, the gains come from the way the text is read rather than from model size.

Weaknesses. The recurrent models are the slowest: Shibin's BiGRU trains about 120 times slower than his baseline (19 minutes against 21 seconds) and is slightly over-confident, with 544 of its 1,681 errors made at a confidence above 90%. For both members short reviews are the hardest subset (5.0% and 5.3% error for the best models), and both error reviews find that mixed sentiment, negation and sarcasm cause most confident mistakes.

Limitations. The labels come from star ratings, not from the text, so some are simply wrong: 5 of the 20 errors we reviewed contradict their own text, and 932 test reviews are misclassified by all three models. Without pretrained knowledge, idioms and sarcasm are hard to learn. We used a single seed per model and a fixed decision threshold of 0.5, and because the two members evaluated on different test sets, differences of a few tenths of a percent between members are not meaningful.

Next steps. Marking the tokens that follow a contrast word or a negation is a cheap change we would test first, on the contrast slice with a McNemar test. We would also try a longer maximum length for long reviews, a small Transformer encoder trained from scratch, an ensemble of the three models with weights chosen on validation, temperature scaling, and several seeds per model.

3.3 Error Review (Shibin Thomas)

We reviewed 20 test reviews misclassified by the best model (BiGRU with attention): the five most confident false positives, the five most confident false negatives, five errors closest to the 0.5 threshold, and five errors from the subset with the highest error rate (short reviews). Each review was read and assigned an error type by hand. Ten of them are shown in Table 5.

Table 5. Examples from the manual review of 20 misclassified test reviews.

#	Label	P(pos)	Error type	Review text (excerpt)
2	neg	1.000	Label noise	<i>Wow love the place and everything is very clean and new! Great place to come and relax worth a try!</i>
4	neg	1.000	Mixed sentiment, implicit verdict	<i>About average so far as steakhouses go. My rib eye was very tasty but a little over cooked. ... Over all I'd say they are more for show than content. If you want to have a great high end steak I'd recommend Ruth Chris...</i>
6	pos	0.000	Label noise (edited review)	<i>EDIT: They really did change the service up since I last posted this. Horrible service. Used to be my favorite pizza in the city...</i>
9	pos	0.001	Sentiment target confusion	<i>Last night several parents came in with over 15 children ... The bartender expressed that it was not a place to have children running around...</i>
13	pos	0.498	Mixed sentiment, humour	<i>Their alcohol selection sucks ... dreamcrushed my cocktail dreams. ... However, the burger was nice, very juicy and rare.</i>
14	neg	0.502	Implicit negativity, domain shift	<i>Overly sentimental, non stop references to her kids, how blessed she is ... too many covers from elite performers...</i>
15	pos	0.497	Idiom	<i>the pizza was to my door in 15 minutes! ... the cookie's are to die for.</i>
16	neg	0.999	Faint praise	<i>Standard take out and it's cheap. Service is fast and friendly. I go here when I'm too lazy to walk to Zaw's...</i>
18	pos	0.002	Complex negation	<i>despite still not digging their ordering process, their food is just too good to disrespect with a 2 star review.</i>
19	pos	0.002	Sarcasm	<i>For being a DUMP, should expect much more. Flies, stink, garbage, dirt ... Keepin it real dumpy!</i>

Source: task2_sentiment/shibin_thomas/failure_analysis.md

Table 6. Error types among the 20 reviewed errors.

Error type	Count
Mixed sentiment (praise and complaints in the same review)	7
Label noise (text contradicts the star rating)	5
Faint praise or implicit negativity	3
Complex negation, target confusion, truncation, idiom, sarcasm	1 each

Five of the 20 errors are not really model errors, because the prediction agrees with the text and the star rating does not. The most common error the model could learn to avoid is mixed sentiment, where the verdict sits in the clause after “but” or “however”. As a testable fix we propose to prefix every token after the last contrast word with a marker, retrain the BiGRU with only this change, and compare the error rate on

the contrast slice (currently 4.87%) with a McNemar test on the same test set.

3.4 Error Review (Denisha Ketan Tank)

Denisha reviewed 20 errors of her best model (BiGRU) chosen with the same scheme: five confident false positives, five confident false negatives, five near-threshold errors and five errors from one slice. Table 7 shows ten of them.

Table 7. Examples from Denisha's manual review of 20 misclassified test reviews.

Test idx	Label	P(pos)	Group	Error type	Review text (excerpt)
209813	neg	1.000	Confident FP	Advice, implicit negation	<i>Save your \$300 initial consultation and go to a Macy's personal shopper--they are free.</i>
175758	neg	1.000	Confident FP	No context	<i>Yup</i>
174044	neg	1.000	Confident FP	Label noise	<i>I love this place every thing about it from the great help to the great quality of the furniture wow that is all I have to say</i>
204270	pos	0.000	Confident FN	Negation	<i>Not expensive..</i>
465105	pos	0.000	Confident FN	Negation and contrast	<i>False advertising! John ISN'T grouchy!</i>
198250	pos	0.000	Confident FN	Slang, negative words in a positive review	<i>went for breakfast. avoided the horrible buffet crowds. kobe short ribs. boo-yah.</i>
505121	neg	0.500	Near threshold	Mixed sentiment	<i>The massage was nice, but the mani/pedi wasn't so great. I have a few nicks on my fingers because my cuticles and the skin around my nails weren't softened enough ...</i>
424220	neg	0.500	Near threshold	Non-English text	<i>Cette petite cantine vietnamienne toujours occup\u00e9e m'a longtemps intrigu\u00e9e.... A tort ! Je passerai d\u00e9sormais mon chemin. C'est du rechauff\u00e9, trop ...</i>
43253	neg	1.000	Slice (medium)	Mixed sentiment, late verdict	<i>Friendly service and a funky throwback ambiance make this place a fun alternative to delicious food. I love a good hot dog, but I'm not convinced that Ted's serves ...</i>
233607	neg	1.000	Slice (medium)	Possible label noise	<i>Complete Madness!!! Well, the spicy chicken is at least. A very simple Asian menu of rice, choice of meat and salad. But the salad dressing is amazing, the spicy ...</i>

Source: task2_sentiment/member_denisha/failure_analysis.md (branch denisha-lab1),
task2_sentiment/member_denisha/outputs/error_review.json (branch denisha-lab1)

Denisha's review finds the same broad pattern as Shibin's. Very short reviews (“Yup”) give the model almost nothing to work with, local negation (“Not expensive”) is read word by word, and long reviews that mix praise and criticism are judged by their most frequent sentiment words rather than by the final verdict. She also found one non-English review and two probable label errors. Her proposed fixes are negation-aware features, an attention or sentence-level pooling layer that can weight the concluding sentences, and an abstention analysis for extremely short reviews.

3.5 Evidence

Evidence	Location in the repository
Shibin: Configurations	task2_sentiment/shibin_thomas/configs
Shibin: Training and evaluation logs	reproducibility/raw_logs/task2_sentiment/shibin_thomas/yelp3_20261001-182849
Shibin: Run manifest	reproducibility/manifests/task2_sentiment/shibin_thomas/yelp3_20261001-182849.json
Shibin: All metrics for every model	task2_sentiment/shibin_thomas/metrics_report.csv
Shibin: ROC, PR, reliability and slice plots	task2_sentiment/shibin_thomas/outputs/yelp3_20261001-182849
Shibin: Learned embedding neighbours	task2_sentiment/shibin_thomas/outputs/yelp3_20261001-182849/embedding_neighbours.md
Shibin: Checkpoints	task2_sentiment/shibin_thomas/checkpoints/yelp3_20261001-182849
Shibin: Results and analysis	task2_sentiment/shibin_thomas/results.md
Shibin: Executed notebook	task2_sentiment/shibin_thomas/src/task2_yelp_sentiment.ipynb
Denisha: Configuration	task2_sentiment/member_denisha/config.yaml (branch denisha-lab1)
Denisha: Training log	reproducibility/raw_logs/task2_denisha.log (branch denisha-lab1)
Denisha: Run manifest	reproducibility/manifests/task2_denisha.json (branch denisha-lab1)
Denisha: All metrics for every model	task2_sentiment/member_denisha/metrics_report.csv (branch denisha-lab1)
Denisha: Data analysis	task2_sentiment/member_denisha/outputs/data_analysis.json (branch denisha-lab1)
Denisha: Error review records	task2_sentiment/member_denisha/outputs/error_review.json (branch denisha-lab1)
Denisha: Checkpoints	task2_sentiment/member_denisha/checkpoints (branch denisha-lab1)
Denisha: Results	task2_sentiment/member_denisha/results.md (branch denisha-lab1)
Denisha: Notebook	task2_sentiment/member_denisha/Task2_Sentiment_Colab.ipynb (branch denisha-lab1)

4 Task 3: CycleGAN Style Transfer between Monet Paintings and Photos

The goal is unpaired translation between 300 Monet paintings (domain A) and 7,038 photos (domain B) with a CycleGAN [3] trained from random initialisation. Shibin's generators are ResNet encoder-decoders with nine residual blocks; upsampling uses nearest-neighbour resizing followed by a convolution instead of transposed convolutions, to avoid checkerboard artifacts. The discriminators are 70x70 PatchGANs [11]. Training combines a least-squares adversarial loss [12], a cycle-consistency loss (weight 10) and an identity loss (weight 5). The Kaggle score is computed with the instructor's script: FID and the script's MiFID on the first 300 sorted images in each direction, averaged, and the final score is the mean of the two values (lower is better). Besides FID [13] we report KID [14], generative precision and recall [15], density and coverage, LPIPS [16], the cycle-reconstruction error and a content similarity score. The submitted images are the unedited outputs of the trained generators; the pretrained Inception and LPIPS networks are only used to measure them.

Denisha trained an independent CycleGAN with the same ResNet-9 generator depth but transposed-convolution upsampling, a PatchGAN discriminator with four stride-2 layers, an identity weight of 2.5, 60 epochs (30 constant and 30 with linear decay), gradient clipping at 5.0 and fp16 mixed precision on an RTX 4090. Her submission is the team's best Kaggle entry.

4.1 Model Comparison

Table 8. Task 3 comparison. Directional metrics use the 300 scored images; the Kaggle score is $(\text{FID} + \text{MiFID}) / 2$. Shibin's run `cyclegan_v1_20261001-201801`. In Denisha's column, two values are given as photo to Monet / Monet to photo.

	Shibin: photo to Monet	Shibin: Monet to photo	Denisha Ketan Tank (CycleGAN)
Architecture	Generators: ResNet-9 (c7s1-64, two stride-2 convs, 9 residual blocks, two resize-conv layers, tanh), InstanceNorm, reflection padding, 11.38M parameters each. Discriminators: 70x70 PatchGAN, 2.76M each.		Generators: ResNet-9 with transposed-convolution upsampling, InstanceNorm, 11.38M parameters each. Discriminators: 70x70 PatchGAN, 2.76M each.
Hyperparameters	LSGAN, cycle weight 10, identity weight 5; Adam 2e-4 (0.5, 0.999); 20 epochs constant LR then 20 epochs linear decay, 2,000 unpaired pairs per epoch; batch 4; image pool 50; resize 286, random crop 256, flip		LSGAN, cycle weight 10, identity weight 2.5; Adam 2e-4 (0.5, 0.999); 30 epochs constant LR then 30 epochs linear decay, 2,000 pairs per epoch; batch 4; replay buffer 50; resize 286, crop 256, flip; gradient clip 5.0; fp16 AMP
FID	118.07	117.34	98.36 / 103.23
MiFID (instructor script)	0.4105	0.4330	0.4069 / 0.4189
KID	0.0197	0.0362	0.0057 / 0.0174 *
Precision (k = 3)	0.267	0.607	0.060 / 0.153 *
Recall (k = 3)	0.640	0.320	0.027 / 0.000 *
Density / coverage (k = 5)	0.221 / 0.523	0.836 / 0.757	not reported
Cycle reconstruction L1	0.0622	0.0531	0.0457 / 0.0389 **
LPIPS, input vs translation	0.451	0.366	not reported
LPIPS, input vs reconstruction	0.384	0.468	0.217 / 0.284
Content cosine similarity	0.750	0.794	0.305 / 0.465 ***
Kaggle file: FID / MiFID (average)	117.7069 / 0.4217, score 59.06		100.7938 / 0.4129, score 50.60
Kaggle leaderboard: score / rank	submitted (scores -59.06); team best entry -50.6033, rank 34		-50.6033 / 34 (team best entry)
Final epoch losses: G / D_A / D_B	4.458 / 0.089 / 0.139		3.098 / 0.084 / 0.144
Final epoch cycle A / B, identity A / B	0.096 / 0.120, 0.108 / 0.110		0.153 (A+B), 0.157 (A+B)
Gradient norm G (mean / max), D (mean / max)	48.3 / 727, 17.1 / 342		logged as NaN (see text)
Non-finite steps / total steps	0 / 20,000		0 / 30,000
Training time / throughput / peak GPU memory	91.2 min / 36.0 images/s / 19,922 MB		68.4 min / 58.5 images/s / 4,816 MB
Total parameters	28,285,832		28,285,832
Human audit (2 raters, 30 samples)	Pending		Pending
Hardware	NVIDIA GeForce RTX 5090		NVIDIA GeForce RTX 4090
Checkpoints (SHA-256)	G_BA.pt e00ed3819109	G_AB.pt 4185c02deae2	340 MB, stored on Google Drive

Source: `task3_gan/shibin_thomas/full_metrics_report.csv`, `task3_gan/shibin_thomas/submission.csv`, `task3_gan/shibin_thomas/kaggle_leaderboard.json`, `task3_gan/member_denisha/outputs/submission_metrics_official.json` (branch `denisha-lab1`), `task3_gan/member_denisha/outputs/metrics_A2B.json` (branch `denisha-lab1`), `task3_gan/member_denisha/outputs/metrics_B2A.json` (branch `denisha-lab1`), `task3_gan/member_denisha/outputs/metrics.json` (branch `denisha-lab1`), `task3_gan/member_denisha/results.md` (branch `denisha-lab1`)

* Denisha's KID and precision/recall were computed with torch-fidelity against all 7,038 photos for Monet to photo, so they are not on exactly the same reference set as Shibin's. ** Denisha's cycle L1 was recorded on the $[-1, 1]$ pixel scale (0.0914 and 0.0778) and is shown here divided by 2 to match Shibin's $[0, 1]$ scale. *** Denisha's content cosine is computed on a different feature representation from Shibin's Inception features, so only the within-member comparison of directions is meaningful.

4.2 Training Behaviour and Results

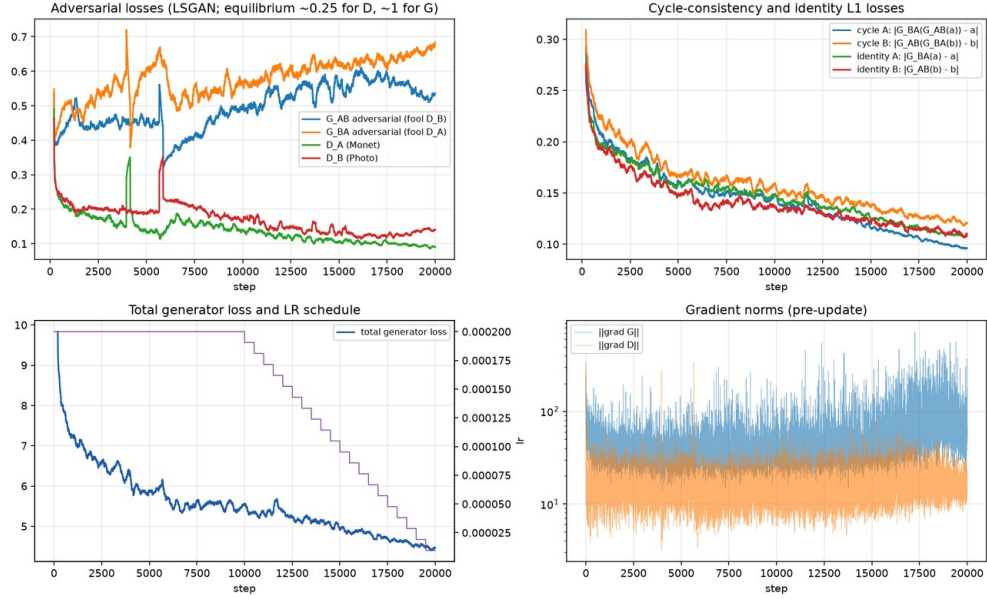


Figure 4. Task 3 adversarial, cycle and identity losses, total generator loss with the learning-rate schedule, and gradient norms over 20,000 steps (task3_gan/shibin_thomas/outputs/cyclegan_v1_20261001-201801/loss_curves.png)

Table 9. FID on the 300 scored images during training (monitoring only).

Epoch	5	10	15	20	25	30	35	40
FID, photo to Monet	153.2	145.4	147.1	128.7	130.2	119.0	123.5	121.6
FID, Monet to photo	187.7	155.2	169.5	128.6	119.7	121.8	121.3	118.1

Source: reproducibility/raw_logs/task3_gan/shibin_thomas/cyclegan_v1_20261001-201801/epochs.csv

Both discriminators settle below the least-squares balance point of 0.25 and keep improving slowly, while the generators' adversarial losses rise; by the end the Monet discriminator (loss 0.089) is clearly winning, a sign that it has started to memorise the 300 paintings. The cycle and identity losses fall smoothly over the whole run. FID improves quickly during the first 25 epochs and then stays within about 4 points of 120, which is within the noise of a 300-image FID. No training step produced a NaN or infinite value.

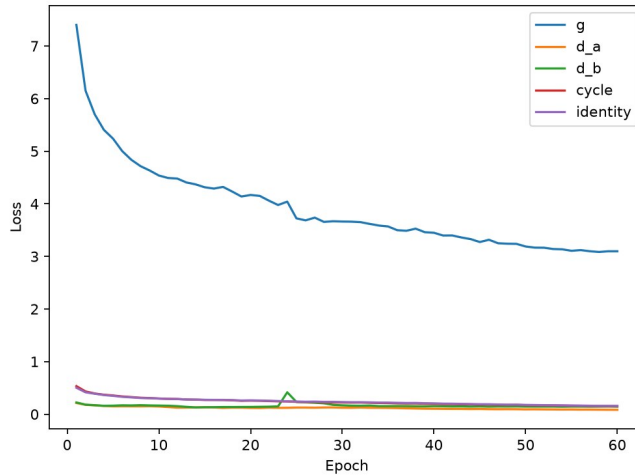


Figure 5. Task 3 generator, discriminator, cycle and identity losses per epoch (Denisha Ketan Tank) (task3_gan/member_denisha/outputs/loss_curves.png)

Denisha's run shows the same balance: her discriminators finish at 0.084 (Monet) and 0.144 (photo), almost exactly the values of Shibin's run, while the cycle and identity losses decrease throughout. Her gradient-norm statistics were written as NaN in the metrics file even though the explicit NaN counter is zero; we report this as a logging defect rather than a training failure.



Figure 6. Photo to Monet. Top row: input photos; middle: generated paintings; bottom: reconstructions (task3_gan/shibin_thomas/outputs/cyclegan_v1_20261001-201801/final_samples_B2A.png)



Figure 7. Monet to photo. Top row: input paintings; middle: generated photos; bottom: reconstructions (task3_gan/shibin_thomas/outputs/cyclegan_v1_20261001-201801/final_samples_A2B.png)

4.3 Discussion

Comparison. Denisha's CycleGAN reaches a clearly better FID (98.4 and 103.2 against 118.1 and 117.3, average 100.8 against 117.7), which gives the team's Kaggle score of -50.60 and rank 34. The two runs differ mainly in the training recipe: 60 instead of 40 epochs, a lower identity weight (2.5 instead of 5), which lets the generator move further from the input colours, and transposed-convolution upsampling. This agrees with the weaknesses Shibin's run showed (FID flat after epoch 30 and weak Monet style) and with the changes in his prepared second configuration. Denisha's run also reconstructs inputs more closely (cycle L1 0.046 and 0.039 on the $[0, 1]$ scale against 0.062 and 0.053, LPIPS 0.22 and 0.28 against 0.38 and 0.47).

Strengths. Both trainings were stable from start to finish and no generator collapsed. The cycle constraint works: reconstructions keep the layout, objects and colours of the inputs (cycle L1 of 0.05 to 0.06), and photo-to-Monet translations preserve the content of the scene (content cosine 0.75) while moving the

colours towards Monet's palette. Our implementation of the scoring protocol matches the instructor's script, so the submitted numbers can be verified.

Weaknesses. Only 27% of the generated paintings fall inside the feature manifold of real Monets (precision 0.27). The main reasons are a regular stippled texture that replaces Monet's brush strokes and the washing out of dark photos. In the other direction the generated photos cover only a third of the photo distribution (recall 0.32) and still look painted. The Monet discriminator over-fits the small training set.

Limitations. FID on 300 images has high variance, each member trained a single seed on different GPUs, and some secondary metrics were computed with different tools (see the notes under Table 8). The script's MiFID is a mean cosine distance between unrelated real and generated images and hardly changes between models, so the Kaggle score is driven almost entirely by FID. The two-rater human audit is still being completed.

Next steps. Based on these findings we prepared a second configuration (`configs/cyclegan_v2.yaml`) with differentiable augmentation for the discriminators [18], an exponential moving average of the generator weights, a lower identity weight of 2.5, 60 epochs, and selection of the exported epoch on held-out photos. A multi-scale discriminator and a low-frequency content loss are further options.

4.4 Failure Analysis (Shibin Thomas)

We ranked all translations by cycle error, by content similarity and by the distance to the nearest real image of the target domain, and inspected the worst four images for each criterion. In each figure below the columns show the input (top), the translation (middle) and the reconstruction (bottom).



Figure 8. Dark and night photos translated to Monet style
(`task3_gan/shibin_thomas/outputs/cyclegan_v1_20261001-201801/failure_B2A_nn_dist.png`)

Dark photos. Low-light scenes such as a river at dusk or a camel silhouette against a sunset are turned into bright, washed-out images covered in coloured speckles, and silhouettes lose their edges. These are the least realistic outputs. The Monet set contains almost no night scenes, so the generator brightens every image. Colour augmentation of the discriminator's inputs is the fix we included in the second configuration.

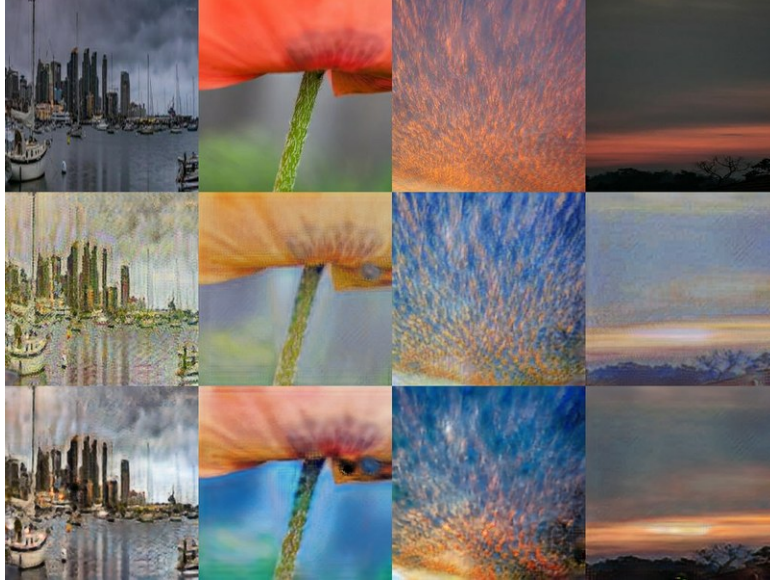


Figure 9. Photos with the highest cycle-reconstruction error
(task3_gan/shibin_thomas/outputs/cyclegan_v1_20261001-201801/failure_B2A_cycle_l1.png)

Recoloured warm tones. Strong reds and oranges are replaced by Monet's blues and lilacs: an orange sky becomes blue and a red poppy turns pale orange. The reconstructions nevertheless recover the original colours, which means the generator hides the information in a faint signal that the cycle loss can read back, an effect described by Chu et al. [17]. When this hidden signal is not enough, the cycle error is highest (0.13 to 0.17, about 2.5 times the average).



Figure 10. Monet paintings with the lowest content similarity after translation to photos
(task3_gan/shibin_thomas/outputs/cyclegan_v1_20261001-201801/failure_A2B_content_cos.png)

Foggy paintings. Pale, low-contrast paintings of fog on the Thames become saturated orange sunsets or turquoise storms, and a thin poplar tree disappears. Photos are rarely foggy, so the generator invents contrast and colour. We also observed a regular stipple texture in most photo-to-Monet outputs and visible brush strokes and signatures in most Monet-to-photo outputs. The full analysis of all five failure types is in failure_analysis.md.

4.5 Evidence

Evidence	Location in the repository
Shibin: Configuration	task3_gan/shibin_thomas/configs/cyclelegan_v1.yaml
Shibin: Training and evaluation logs	reproducibility/raw_logs/task3_gan/shibin_thomas/cyclelegan_v1_20261001-201801
Shibin: Run manifest	reproducibility/manifests/task3_gan/shibin_thomas/cyclelegan_v1_20261001-201801.json
Shibin: Submitted predictions	task3_gan/shibin_thomas/outputs
Shibin: Kaggle submission file	task3_gan/shibin_thomas/submission.csv
Shibin: Plots and sample grids	task3_gan/shibin_thomas/outputs/cyclelegan_v1_20261001-201801
Shibin: Generator checkpoints	task3_gan/shibin_thomas/checkpoints/cyclelegan_v1_20261001-201801
Shibin: Human audit material	task3_gan/shibin_thomas/outputs/human_audit
Shibin: Results and analysis	task3_gan/shibin_thomas/results.md
Shibin: Failure analysis	task3_gan/shibin_thomas/failure_analysis.md
Shibin: Executed notebook	task3_gan/shibin_thomas/src/task3_cyclelegan.ipynb
Denisha: Configuration	task3_gan/member_denisha/config_competition.yaml (branch denisha-lab1)
Denisha: Training log	reproducibility/raw_logs/task3_denisha.log (branch denisha-lab1)
Denisha: Run manifest	reproducibility/manifests/task3_denisha.json (branch denisha-lab1)
Denisha: Official evaluation	task3_gan/member_denisha/outputs/submission_metrics_official.json (branch denisha-lab1)
Denisha: Kaggle submission file	task3_gan/member_denisha/outputs/submission.csv (branch denisha-lab1)
Denisha: Generated images (photo to Monet)	task3_gan/member_denisha/outputs/images.zip (branch denisha-lab1)
Denisha: Loss history and curves	task3_gan/member_denisha/outputs/history.json (branch denisha-lab1)
Denisha: Human audit sheet	task3_gan/member_denisha/outputs/human_audit.csv (branch denisha-lab1)
Denisha: Results	task3_gan/member_denisha/results.md (branch denisha-lab1)
Denisha: Notebook	task3_gan/member_denisha/Task3_VSCode.ipynb (branch denisha-lab1)

5 Pre-Submission Checklist

Item	Status
Every member's folder exists under all three tasks with code, configuration, logs and results.md	Shibin: branch shibin-lab1; Denisha: member_denisha folders on branch denisha-lab1
No two members share the same architecture and hyperparameters	Confirmed (Section 2 to 4); the two TextCNNs share layer sizes but differ in batch size, dropout, schedule, epochs and precision
Task 1 uses no prebuilt Transformer or attention modules	Done for both members
Task 2 uses no pretrained embeddings or language models; three models per member	Done for both members
All required metrics are reported for every model in Tasks 1 to 3	Done (Tables 1, 3 and 8); a few secondary values that a member did not record are marked
Task 3 Kaggle submission made and leaderboard rank recorded	Done: team score -50.6033, rank 34
Task 3 two-rater human audit (30 samples)	Pending: ratings from both raters
Comparison tables with architecture, hyperparameters and metrics for all three tasks	Done (Tables 1, 3 and 8)
Raw logs and manifests committed and unmodified	Done

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [2] R. Eldan and Y. Li. TinyStories: How small can language models be and still speak coherent English? *arXiv:2305.07759*, 2023.
- [3] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *ICCV*, 2017.
- [4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.
- [5] R. Xiong et al. On layer normalization in the Transformer architecture. In *ICML*, 2020.
- [6] X. Zhang, J. Zhao, and Y. LeCun. Character-level convolutional networks for text classification. In *NeurIPS*, 2015.
- [7] Y. Kim. Convolutional neural networks for sentence classification. In *EMNLP*, 2014.
- [8] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov. Bag of tricks for efficient text classification. In *EACL*, 2017.
- [9] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.
- [10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.
- [11] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks. In *CVPR*, 2017.
- [12] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley. Least squares generative adversarial networks. In *ICCV*, 2017.
- [13] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *NeurIPS*, 2017.
- [14] M. Bińkowski, D. J. Sutherland, M. Arbel, and A. Gretton. Demystifying MMD GANs. In *ICLR*, 2018.
- [15] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila. Improved precision and recall metric for assessing generative models. In *NeurIPS*, 2019.
- [16] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [17] C. Chu, A. Zhmoginov, and M. Sandler. CycleGAN, a master of steganography. *NeurIPS Workshop*, 2017.

- [18] S. Zhao, Z. Liu, J. Lin, J.-Y. Zhu, and S. Han. Differentiable augmentation for data-efficient GAN training. In *NeurIPS*, 2020.